

Termin na wysłanie zadania w systemie UPEL:

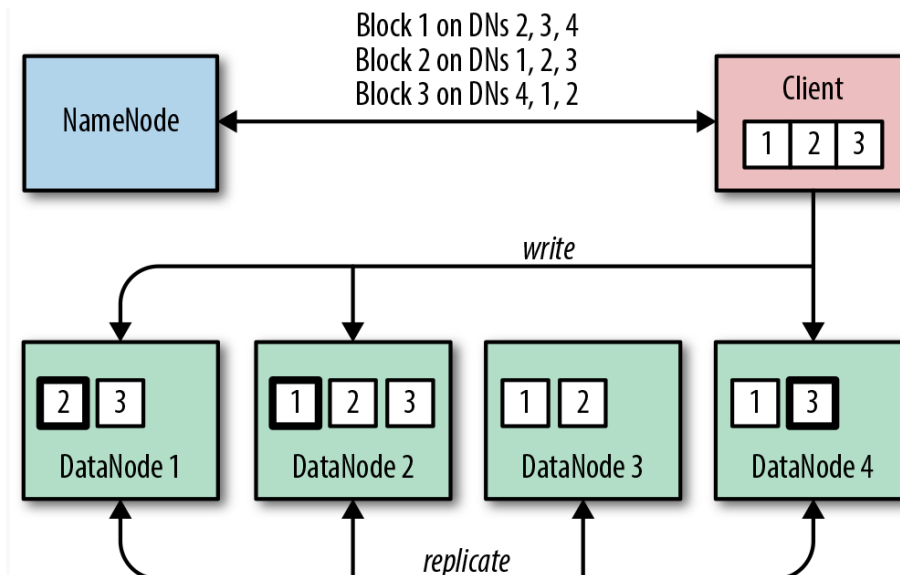
poniedziałek, 1 czerwca 2026, godz. 13:00 (wszystkie grupy).

Tier 1 (3 pkt)

- Przygotuj i zaprezentuj zadanie umieszczone w Jupyter Notebook
- Obliczenie liczby Pi + równoległe sortowanie przez scalanie (szczegóły w noteboku)

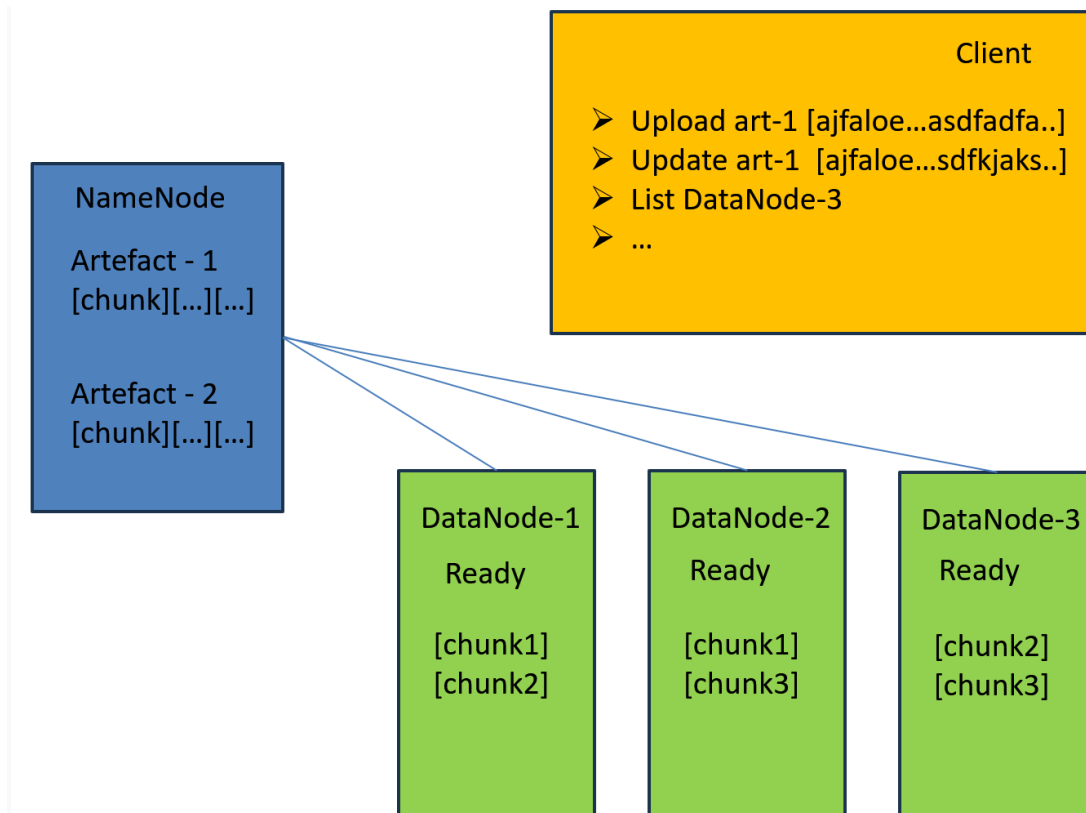
Tier 2 (6-10 pts)

- Zaimplementuj rozproszony system do przechowywania artefaktów na bazie rozproszonego systemu plików
- Schemat HDFS poniżej



- Artefakt składa się z nazwy i długiego napisu, który jest rozbijany na węzły
- Jest jeden name-node i kilka storage-node - wszystkie węzły to aktorzy
- Całość implementacji może być przedstawiona w *Jupyter Notebook*'u lub jako narzędzie *command-line*
- Aktor name-node dystrybuje artefakty do węzłów storage-node w chunkach/blokach i monitoruje je
- Aktor *NameNode* dystrybuje dane w sensie globalnym i jest nadzorcą metadanych wszystkich artefaktów. *DataNodes* między sobą nie uczestniczą w procesie uzgadniania który chce/może przyjąć kawałek artefakt i który chce/może zrobić kopie
- Dane z klienta są umieszczane bezpośrednio na *DataNodes* (a nie kopiowane na *NameNode* i później kopiowane na *DataNodes*), tak jak na rysunku

- Istnieje kilka kopii takich bloków w systemie (aby poprawić wysoką dostępność i transfer danych - podobnie jak w HDFS)
- Węzły storage-node mogą ulec awarii (zmiana statusu); węzeł name-node śledzi wszystkie fragmenty i zarządza kopiami. Wszystkie dane, operacje i stan systemu powinny być spójne – dotyczy to również wszystkich chunków
- Wszystkie węzły powinny mieć metody dostarczające możliwość wylistowania najważniejszych danych (np. gdzie przechowywane są chunki/bloki)
- Poniżej schemat systemu



- Podczas oddawania zadania proszę o przygotowanie następujących testów. Użytkownik ma możliwość: stworzenia, aktualizacji, skasowania, pobrania artefaktu oraz sytuacji, w której jeden data-node jest kasowany. Operacje powinny być weryfikowane przez wylistowanie stanu węzłów.
- Chciałbym żeby update nie polegał na skasowaniu wszystkiego i stworzeniu nowego artefaktu w każdym wypadku (a w szczególności w sytuacji zmiany jednego znaku w artefakcie)

Punktacja Tier 2:

- Bazowy scenariusz opisany wyżej : 6-7 pkt

- Dołożenie jednego elementu z Ray Actor API o którym nie mówiliśmy na zajęciach (<https://docs.ray.io/en/latest/ray-core/actors.html>): +1-2 pkt
- Dołożenie rozproszenie środowiska (patrz niżej): +1 pkt

Tier 3 (9-10 pkt)

- Przygotuj przykładowy projekt realizujący scenariusz ML w wykorzystaniem modułów Ray AI Runtime
- W projekcie należy wykorzystać w miarę unikalne zbiory danych (raczej nie: MNIST, irysy itp.)
- Należy użyć 3 różnych modułów, i w miarę możliwości przygotować testy/ewaluacje
- Jakiekroki były potrzebne aby przenieść projekt do Ray AIR, jakie były problemy

Punktacja Tier 3:

- Przygotowania i prezentacja projektu: 9 pkt
- Dołożenie rozproszenie środowiska (patrz niżej): +1 pkt

Środowisko rozproszone:

- Przygotowane zadanie należy uruchomić w środowisku rozproszonym (może być elementarne środowisko: 1 head node + 1 worker) – weryfikowane na podstawie dostępu do konsoli Ray.
- Można użyć docker-compose (<https://github.com/MarvinSt/ray-docker-compose>), trzeba tylko podmienić wersję pythona i Ray Framework, oraz być może zmienić środowisko w .env
- Można użyć kubernetes (<https://docs.ray.io/en/latest/cluster/kubernetes/index.html>), klaster lokalny lub w chmurze
- Można też uruchomić w chmurze (<https://docs.ray.io/en/latest/cluster/yms/user-guides/launching-clusters/index.html>) - ta opcja wydaje się być najbardziej złożona